Analyzing data on the level of cognitive processes - ESCoP 2025

Gidon T. Frischkorn, Ruhi Bhanap, and Isabel Courage

Exercise 6: Full Diffusion Decision Model

In this exercise, we will have a look at estimating the parameters from the diffusion decision model using the hierarchical Bayesian implementation of the full Diffusion Decision Model.

Your task is to load one of the data sets we prepared for this exercise, either `exc4_VisualSearch.txt` or `exc5_ColorJudgement.txt`.

The Color Judgement data set contains data of 50 participants that completed a Color Judgement task. In this task participants saw an array of blue and red dots and should decide if the array contained more blue or red dots. There were two difficulty levels, the easy level in which 60% of the dots were either blue or red, and the difficult level in which 52% of the dots were either blue or red.

In this exercise, we implement the four parameter `ddm` and test if participants were biased in responding with one of the two response options (i.e. blue or red). For this we have to specify a `ddm` with the actual responses - that is blue or red - as the response boundaries. Thus, we will recode the `correct_response` variable into a numeric `response` variable, and then estimate the `drift` separately for the target containing more blue or red dots.

Solution

```
# Load required packages
library(bmm)
library(tidyverse)
library(here)
```

1) prepare the data

First, we need to prepare the data so that we can analyze our data with the `ddm` using the `bmm` function. For this we need to include a numeric `response` variable coding blue as `0` and red as `1`. Additionally, we need to ensure that our `rt` is stored in seconds.

```
ddm_data <- read.table(here("data","exc5_ColorJudgementTask.txt"),
                       header = T, sep = ",")

ddm_data$rt <- ddm_data$rt/1000
ddm_data$resp_num <- ifelse(ddm_data$resp == "red",1,0)
ddm_data$target <- ifelse(ddm_data$correct_response == "red","red","blue")
```

2) Specify the `ddm` model object

Next, we need to specify the model object to link which data columns are used to identify the `ezdm` using the `rt` in seconds and the newly coded `response` variable as the required variables

```
ddm_model <- ddm(rt = "rt", response = "resp_num")
```

3) specify the `bmmformula` with

Now, we can consider which parameters are affected by the experimental manipulations. Here we assume different `drift` dependent on the target (i.e. are there more red or blue dots) and the difficulty of the task. I also included the condition effect on the non-decision time. Apart from that I only include fixed and random intercepts for the boundary and relative starting point.

```
ddm_formula <- bmf(
  drift ~ 0 + target:condition + (0 + target:condition || participant_id),
  bound ~ 1 + (1 || participant_id),
  ndt ~ 0 + condition + (0 + condition || participant_id),
  zr ~ 1 + (1 || participant_id)
)

default_prior(ddm_formula, data = ddm_data, model = ddm_model)
```

```
## Warning: Rows containing NAs were excluded from the model.
## Warning: Rows containing NAs were excluded from the model.
## Warning: Rows containing NAs were excluded from the model.
```

```
##                 prior     class                     coef          group resp
##  student_t(3, 0, 2.5)        sd
##  student_t(3, 0, 2.5)        sd                         participant_id
##  student_t(3, 0, 2.5)        sd                 Intercept participant_id
##           cauchy(0,1)         b targetblue:conditioneasy
##           cauchy(0,1)         b targetblue:conditionhard
##           cauchy(0,1)         b  targetred:conditioneasy
##           cauchy(0,1)         b  targetred:conditionhard
##  student_t(3, 0, 2.5)        sd
##  student_t(3, 0, 2.5)        sd                         participant_id
##  student_t(3, 0, 2.5)        sd targetblue:conditioneasy participant_id
##  student_t(3, 0, 2.5)        sd targetblue:conditionhard participant_id
##  student_t(3, 0, 2.5)        sd  targetred:conditioneasy participant_id
##  student_t(3, 0, 2.5)        sd  targetred:conditionhard participant_id
##      normal(-1.5,0.5)         b            conditioneasy
##      normal(-1.5,0.5)         b            conditionhard
##  student_t(3, 0, 2.5)        sd
##  student_t(3, 0, 2.5)        sd                         participant_id
##  student_t(3, 0, 2.5)        sd            conditioneasy participant_id
##  student_t(3, 0, 2.5)        sd            conditionhard participant_id
##  student_t(3, 0, 2.5)        sd
##  student_t(3, 0, 2.5)        sd                         participant_id
##  student_t(3, 0, 2.5)        sd                 Intercept participant_id
##           cauchy(0,1)         b
##         normal(0,0.5) Intercept
##      normal(-1.5,0.5)         b
##         normal(0,0.5) Intercept
##           constant(0) Intercept
##           constant(0) Intercept
##           constant(0) Intercept
##           constant(0) Intercept
##    dpar nlpar   lb   ub       source
##   bound          0           default
##   bound          0      (vectorized)
##   bound          0      (vectorized)
##   drift       <NA> <NA> (vectorized)
```

```
##   drift       <NA> <NA> (vectorized)
##   drift       <NA> <NA> (vectorized)
##   drift       <NA> <NA> (vectorized)
##   drift          0           default
##   drift          0      (vectorized)
##   drift          0      (vectorized)
##   drift          0      (vectorized)
##   drift          0      (vectorized)
##   drift          0      (vectorized)
##     ndt       <NA> <NA> (vectorized)
##     ndt       <NA> <NA> (vectorized)
##     ndt          0           default
##     ndt          0      (vectorized)
##     ndt          0      (vectorized)
##     ndt          0      (vectorized)
##      zr          0           default
##      zr          0      (vectorized)
##      zr          0      (vectorized)
##   drift       <NA> <NA>         user
##   bound       <NA> <NA>         user
##     ndt       <NA> <NA>         user
##      zr       <NA> <NA>         user
##               <NA> <NA>         user
##  sdrift       <NA> <NA>         user
##    sndt       <NA> <NA>         user
##     szr       <NA> <NA>         user
```

4) estimate the model

Finally, we use the `bmm` function to fit the full DDM by passing the `ddm` model object and the `bmmformula`. Again, I saved the model results to save time. The `ddm` can take quite some time for parameter estimation. For this data it took about an hour on my machine.

```
ddm_fit <- bmm(
  formula = ddm_formula,
  data = ddm_data,
  model = ddm_model,
  cores = 4,
  sample_prior = TRUE,
  backend = "cmdstanr",
  file = here("models","ddm_colortask_exc6")
)
```

5) Evalutate results

First, we can have a look at the summary of the fitted `ddm` model object.

```
summary(ddm_fit)
```

```
## Loading required namespace: rstan
```

```
##   Model: ddm(rt = "rt",
##              response = "resp_num")
##   Links: drift = identity; bound = log; ndt = log; zr = logit; sdrift = identity; sndt = identity; s
## Formula: drift ~ 0 + target:condition + (0 + target:condition || participant_id)
##          bound ~ 1 + (1 || participant_id)
```

```
##          ndt ~ 0 + condition + (0 + condition || participant_id)
##          zr ~ 1 + (1 || participant_id)
##          sdrift = 0
##          sndt = 0
##          szr = 0
##    Data: ddm_data (Number of observations: 9944)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~participant_id (Number of levels: 50)
##                                   Estimate Est.Error l-95% CI u-95% CI Rhat
## sd(drift_targetblue:conditioneasy)     0.88      0.10     0.71     1.10 1.00
## sd(drift_targetred:conditioneasy)      0.87      0.10     0.70     1.08 1.01
## sd(drift_targetblue:conditionhard)     0.32      0.05     0.24     0.42 1.00
## sd(drift_targetred:conditionhard)      0.38      0.05     0.30     0.49 1.00
## sd(bound_Intercept)                    0.23      0.03     0.19     0.28 1.00
## sd(ndt_conditioneasy)                  0.28      0.03     0.22     0.35 1.00
## sd(ndt_conditionhard)                  0.17      0.02     0.14     0.22 1.00
## sd(zr_Intercept)                       0.13      0.02     0.09     0.18 1.00
##                                   Bulk_ESS Tail_ESS
## sd(drift_targetblue:conditioneasy)     1105     2121
## sd(drift_targetred:conditioneasy)       991     1693
## sd(drift_targetblue:conditionhard)     1276     2335
## sd(drift_targetred:conditionhard)      1338     1835
## sd(bound_Intercept)                     797     1304
## sd(ndt_conditioneasy)                  1044     1879
## sd(ndt_conditionhard)                  1225     2101
## sd(zr_Intercept)                       1568     2179
##
## Regression Coefficients:
##                                 Estimate Est.Error l-95% CI u-95% CI Rhat
## bound_Intercept                     0.73      0.03     0.67     0.79 1.00
## zr_Intercept                       -0.05      0.02    -0.09     0.00 1.00
## drift_targetblue:conditioneasy     -2.15      0.13    -2.42    -1.89 1.00
## drift_targetred:conditioneasy       2.09      0.13     1.83     2.34 1.00
## drift_targetblue:conditionhard     -0.58      0.05    -0.69    -0.48 1.00
## drift_targetred:conditionhard       0.53      0.06     0.42     0.64 1.00
## ndt_conditioneasy                  -1.03      0.04    -1.11    -0.95 1.01
## ndt_conditionhard                  -0.90      0.03    -0.96    -0.85 1.00
##                                 Bulk_ESS Tail_ESS
## bound_Intercept                      408      869
## zr_Intercept                        2209     2580
## drift_targetblue:conditioneasy       472      875
## drift_targetred:conditioneasy        557      857
## drift_targetblue:conditionhard      1201     1808
## drift_targetred:conditionhard        845     1266
## ndt_conditioneasy                    217      541
## ndt_conditionhard                    715     1223
##
## Constant Parameters:
##                      Value
## sdrift_Intercept      0.00
## sndt_Intercept        0.00
```

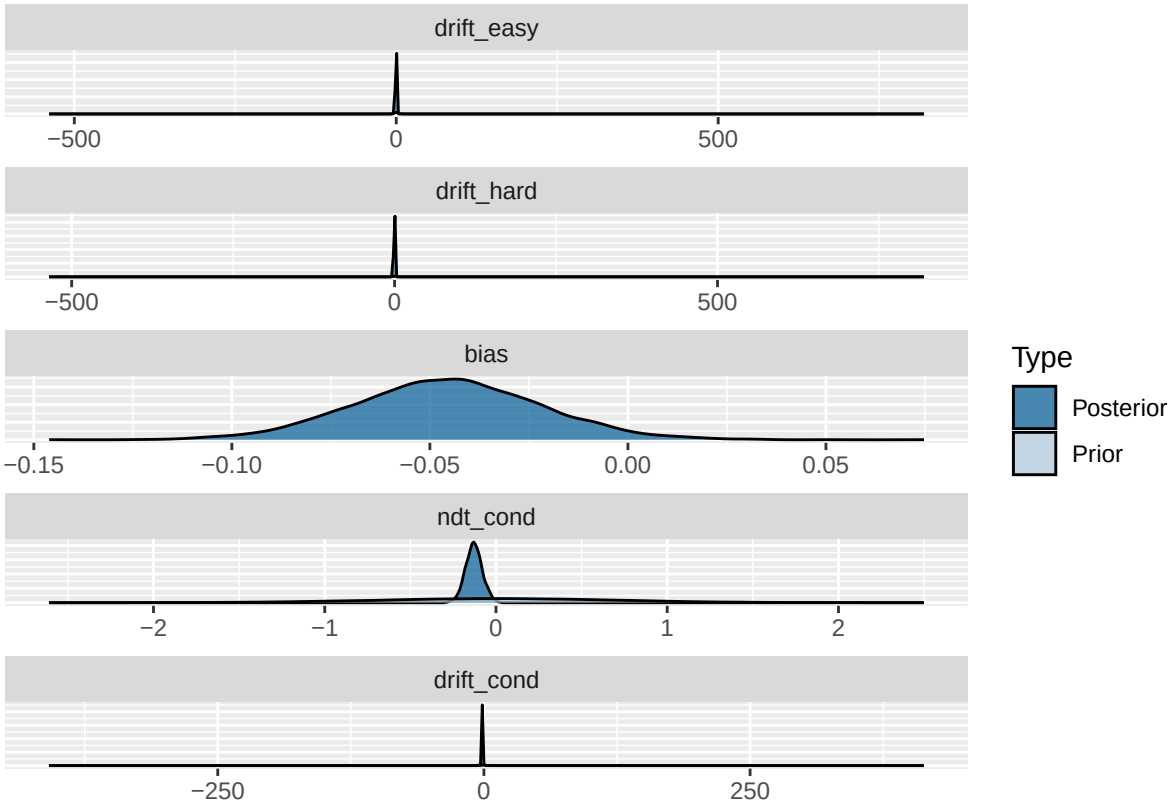
```
## szr_Intercept         0.00
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

Then, we can test some hypothesis:

```
ddm_hypothesis <- c(
  drift_easy = "drift_targetblue:conditioneasy = -drift_targetred:conditioneasy",
  drift_hard = "drift_targetblue:conditionhard = -drift_targetred:conditionhard",
  bias = "zr_Intercept = 0",
  ndt_cond = "ndt_conditioneasy = ndt_conditionhard",
  drift_cond = "(drift_targetblue:conditioneasy - drift_targetred:conditioneasy)/2 =
  (drift_targetblue:conditionhard - drift_targetred:conditionhard)/2"
)

ddm_hypothesis_results <- brms::hypothesis(
  ddm_fit,
  ddm_hypothesis
)

plot(ddm_hypothesis_results)
```




```
ddm_hypothesis_results
```

```
## Hypothesis Tests for class b:
##   Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio Post.Prob Star
## 1 drift_easy    -0.06      0.19    -0.45     0.30      13.34      0.93
## 2 drift_hard    -0.05      0.08    -0.20     0.10      27.74      0.97
```

```
## 3       bias    -0.05      0.02    -0.09     0.00         NA        NA
## 4   ndt_cond    -0.13      0.05    -0.22    -0.03       0.45      0.31    *
## 5 drift_cond    -1.56      0.10    -1.74    -1.36       0.00      0.00    *
## ---
## 'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
## '*': For one-sided hypotheses, the posterior probability exceeds 95%;
## for two-sided hypotheses, the value tested against lies outside the 95%-CI.
## Posterior probabilities of point hypotheses assume equal prior probabilities.
```

```
1/abs(ddm_hypothesis_results$hypothesis$Evid.Ratio)
```

```
## [1] 7.496873e-02 3.604722e-02           NA 2.206984e+00 9.541818e+19
```